

Classification & Three Important Algorithms

Classification is a supervised machine learning problem where the output/target is a category or class.

Examples:

- Email → Spam / Not Spam
- Loan → Approved / Rejected
- Student → Pass / Fail
- Customer → Churn / No Churn
- Disease → Positive / Negative

TWO TYPES:

Binary Classification • 0 / 1 • Yes / No • Pass / Fail	Multiclass Classification • Cat / Dog / Horse • Low / Medium / High
---	--

REGRESSION vs CLASSIFICATION

Regression Input → Model → Numerical value Example: Height → Model → 68.5 kg	Classification Input → Model → Category Example: Income + Age → Model → Loan Approved
---	--

TODAY'S ALGORITHMS:

1	2	3
Logistic Regression Probability-based classification	Decision Tree If-else decision rules	Random Forest Multiple decision trees + voting

MEMORY TRICK:

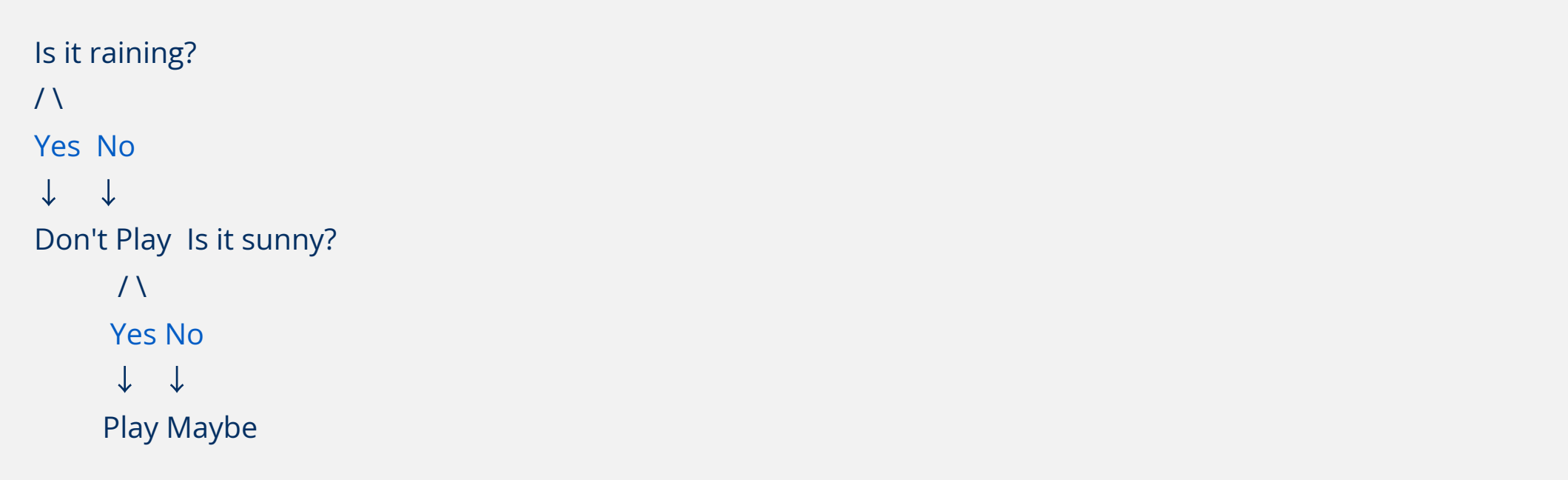
Logistic Regression "How probable is this class?"	Decision Tree "Which rule should I follow?"	Random Forest "What do many trees collectively decide?"
---	---	---

Decision Tree: Learning Like a Human

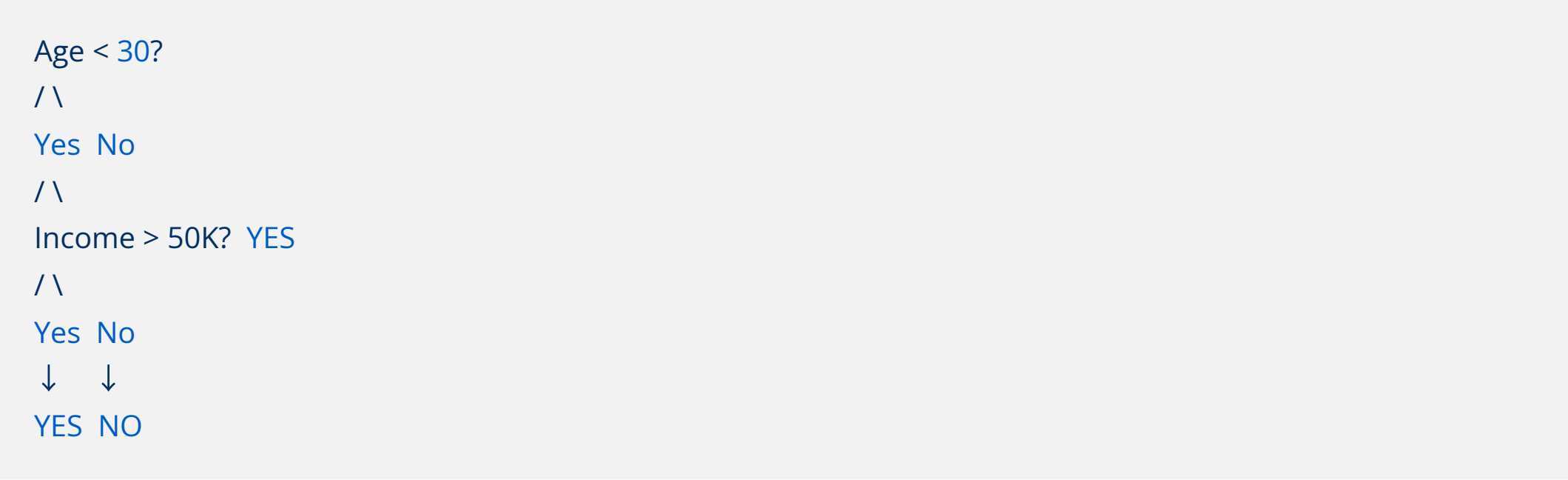
A Decision Tree is a supervised machine learning algorithm that makes predictions by asking a sequence of questions.

It works similarly to how humans make decisions.

EXAMPLE:



ML EXAMPLE — LOAN APPROVAL:



IMPORTANT TERMS:

Root Node First decision/question.	Internal Node A decision/question inside the tree.	Branch Outcome of a decision.
Leaf Node Final prediction.	Depth Number of levels in the tree.	

CORE IDEA:

A Decision Tree repeatedly divides the dataset into groups that become increasingly pure with respect to the target class.

Example:

Before splitting: YES YES NO YES NO NO YES NO

After a good split: YES YES YES YES NO NO NO NO

The second groups are more pure.

How Does a Decision Tree Choose a Split?

The tree does NOT randomly select a question.

It evaluates different features and thresholds and selects a split that improves class separation.

EXAMPLE:

Income	Purchased
20K	No
25K	No
30K	No
60K	Yes
70K	Yes
80K	Yes

Possible question:

Income < 50K?

Income < 50K?

/ \

YES NO

↓ ↓

No Yes

No Yes

No Yes

This is a good split because the classes become separated.

BEFORE SPLIT:

YES + NO are mixed together

↓ High impurity

AFTER GOOD SPLIT:

- Group 1 → Mostly NO
- Group 2 → Mostly YES
- ↓ Lower impurity

The algorithm may evaluate questions such as:

- Age < 30?
- Income < 50K?
- Credit Score < 700?
- Loan Amount < 5 Lakh?

It calculates how good each split is using criteria such as:

- Gini Impurity
- Entropy
- Information Gain



IMPORTANT: Decision Trees do not simply select the feature with the highest correlation. They select splits based on impurity reduction or information gain.

Gini Impurity

Gini Impurity measures how mixed the classes are inside a node.

FORMULA:

$$\text{Gini} = 1 - \sum_{i=1}^C (p_i)^2$$

Where:

p_i = proportion of observations belonging to class i

EXAMPLE:

Suppose a node contains 10 customers.

YES = 8 NO = 2

Therefore:

$P(\text{YES}) = 8/10 = 0.8$ $P(\text{NO}) = 2/10 = 0.2$

Gini:

$$\text{Gini} = 1 - (0.8^2 + 0.2^2) = 1 - (0.64 + 0.04) = 1 - 0.68 = 0.32$$

PURE NODE:

YES YES YES YES YES

$P(\text{YES}) = 1$

$$\text{Gini} = 1 - 1^2 = 0$$

Therefore:

Gini = 0 → Completely pure node

GENERAL IDEA:

Lower Gini ↓ Higher purity ↓ Better class separation	Higher Gini ↓ More mixed classes
---	--

📌 **IMPORTANT INTERVIEW QUESTION:** What does Gini = 0 mean?
Answer: The node is completely pure; all observations belong to one class.

Entropy & Information Gain

Entropy is another measure of impurity/uncertainty.

It measures how uncertain the class distribution is.

FORMULA:

$$\text{Entropy} = - \sum_{i=1}^C p_i \log_2(p_i)$$

PURE NODE:

YES YES YES YES

There is no uncertainty.

Entropy = 0

50/50 NODE:

YES YES NO NO

P(YES) = 0.5 P(NO) = 0.5

Entropy = 1

Therefore:

Entropy = 0

↓ Completely pure

Higher Entropy

↓ More uncertainty

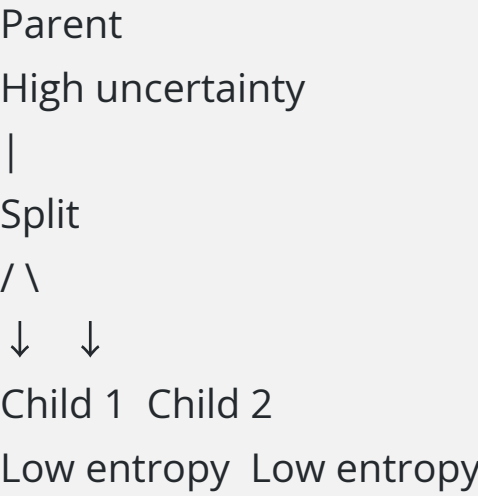
INFORMATION GAIN:

Information Gain tells us how much uncertainty is reduced after performing a split.

FORMULA:

$$\text{Information Gain} = \text{Entropy}(\text{Parent}) - \text{Weighted Entropy}(\text{Children})$$

VISUAL:



A good split:

High parent uncertainty → Low child uncertainty

GINI vs ENTROPY:

Gini

Measures impurity.

Entropy

Measures uncertainty.

Both can be used to select good classification splits.

PYTHON:

```
DecisionTreeClassifier(  
    criterion="gini"  
)
```

OR

```
DecisionTreeClassifier(  
    criterion="entropy"  
)
```

Decision Tree Training & Overfitting

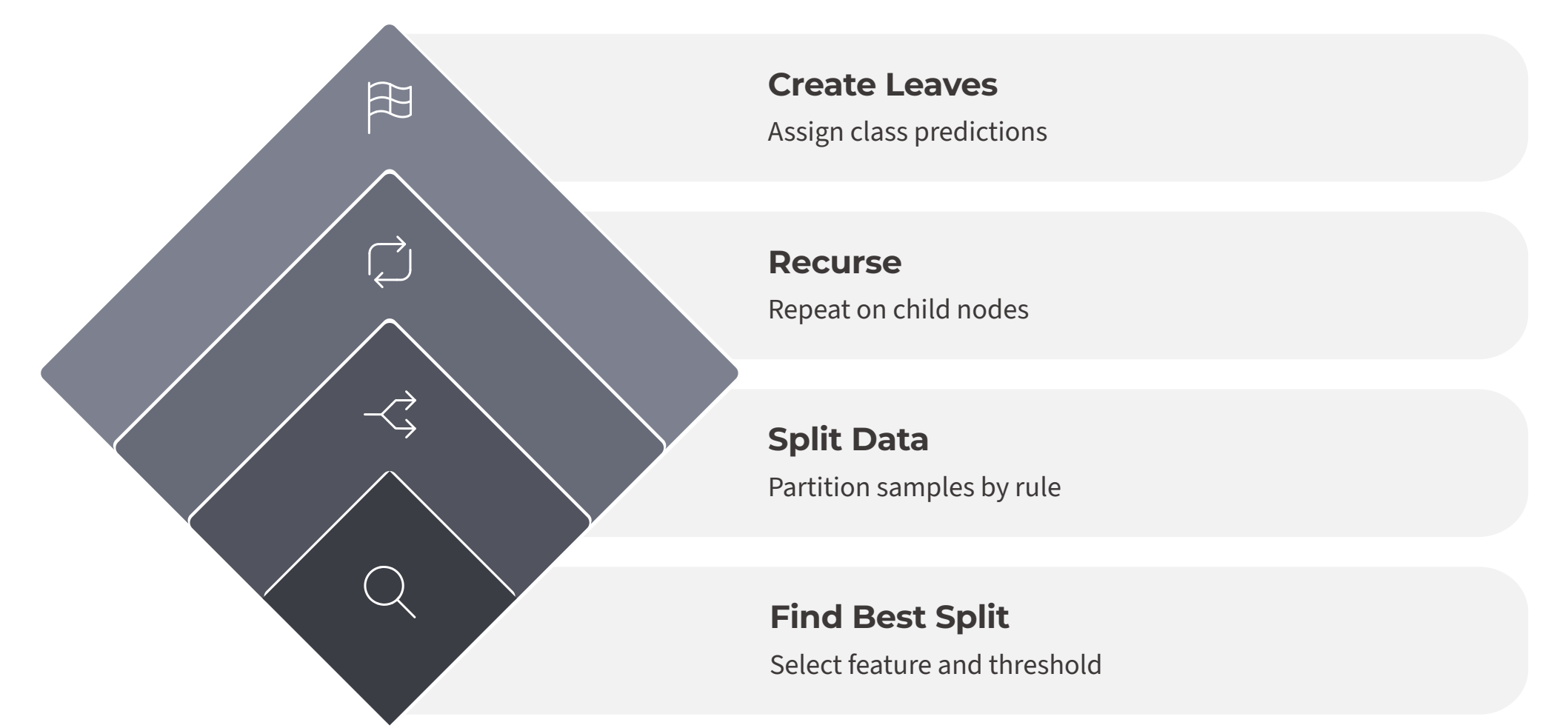
PYTHON:

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,
    random_state=42
)

model.fit(X_train, y_train)
prediction = model.predict(X_test)
```

TRAINING PROCESS:



Training Data → Find best feature/threshold → Split data → Find next best split → Continue recursively → Create leaf nodes → Make prediction

PROBLEM — OVERFITTING:

If the tree becomes extremely deep:

Root

/ \

/ \

/ \

/ \

/ \

Many branches Many branches

The model may memorize the training data.

Example:

Training Accuracy = 100%

Testing Accuracy = 70%

This may indicate overfitting.

CONTROLLING TREE COMPLEXITY:

1

max_depth
Controls maximum depth.
max_depth=5

2

min_samples_split
Minimum samples required to split a node.
min_samples_split=10

3

min_samples_leaf
Minimum samples allowed in a leaf.
min_samples_leaf=5

📌 **IMPORTANT:** Too shallow → Underfitting | Too deep → Overfitting | Appropriate complexity → Better generalization

Random Forest: Many Trees

A single Decision Tree can be sensitive to the training data.

Small changes in data can produce a different tree.

Solution: Build MANY Decision Trees. This is called **RANDOM FOREST**.

VISUAL:



EXAMPLE:

- Tree 1 → YES
- Tree 2 → YES
- Tree 3 → NO
- Tree 4 → YES
- Tree 5 → NO

Votes: YES = 3 NO = 2

Final prediction: YES

WHAT IS RANDOM FOREST?

Random Forest is an ensemble learning algorithm that combines multiple Decision Trees.

WHY "RANDOM"?

Two important sources of randomness:

1. Bootstrap Sampling

Different trees are trained using different samples drawn from the training data, generally with replacement.

2. Random Feature Selection

Different subsets of features can be considered when creating splits.

CORE IDEA:



Random Forest: Math + Python

CLASSIFICATION:

Random Forest generally uses majority voting.

Example:

Tree predictions:

T1 = 1 T2 = 1 T3 = 0 T4 = 1 T5 = 0

Mode:

Mode(1,1,0,1,0) = 1

Final prediction: Class 1

REGRESSION:

Random Forest generally averages predictions.

Formula:

$$\text{Prediction} = \frac{T_1 + T_2 + T_3 + \dots + T_n}{n}$$

PYTHON:

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,
    max_depth=5,
    random_state=42
)

rf.fit(X_train, y_train)
prediction = rf.predict(X_test)
print(prediction)
```

PREDICT PROBABILITY:

```
probability = rf.predict_proba(X_test)
print(probability)
```

IMPORTANT PARAMETERS:

n_estimators Number of trees.	max_depth Maximum depth of each tree.	max_features Number of features considered for a split.
min_samples_split Minimum samples required to split a node.	min_samples_leaf Minimum samples required in a leaf.	

DECISION TREE vs RANDOM FOREST:

Decision Tree

- One tree
- Easy to visualize
- Easy to interpret
- Can overfit easily

Random Forest

- Many trees
- Usually more robust
- Usually less prone to overfitting
- Harder to interpret
- More computationally expensive

Logistic Regression

IMPORTANT: Despite its name, Logistic Regression is primarily used for **CLASSIFICATION**.

EXAMPLE: Hours Studied → Pass / Fail

STEP 1 — LINEAR SCORE:

$$z = b_0 + b_1x$$

Problem: z can have any value: -10 -3 0 2 10

This is NOT a probability. Probability must be: $0 \leq P \leq 1$

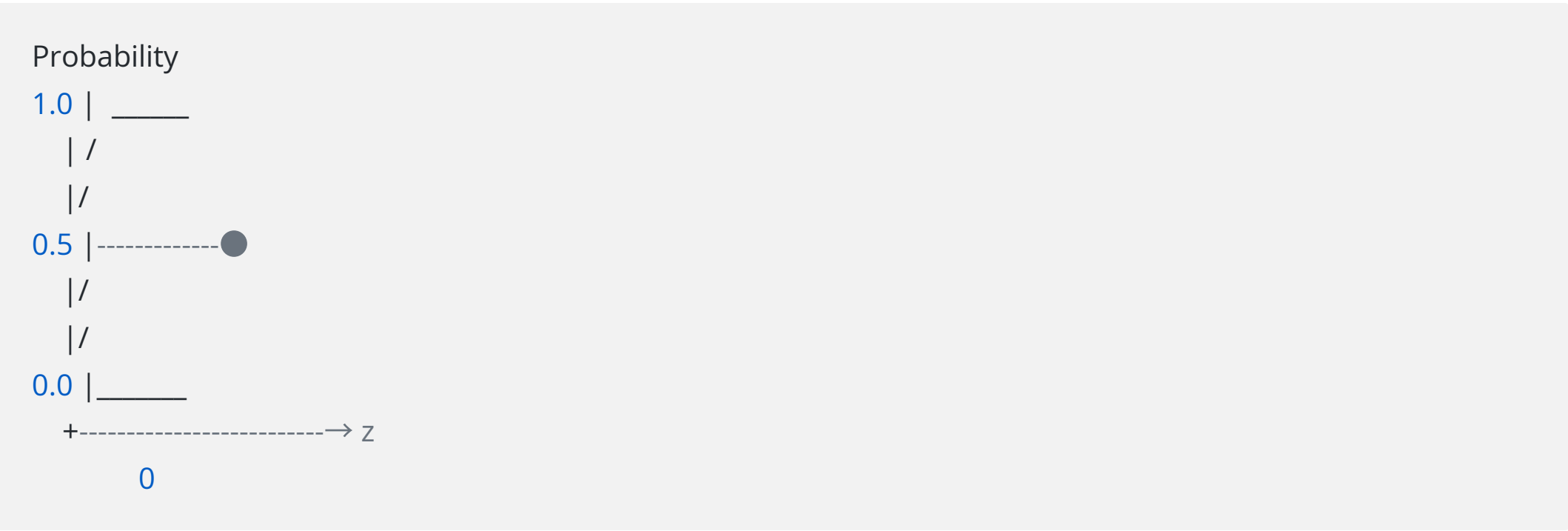
STEP 2 — SIGMOID FUNCTION:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

EXAMPLES:

- If z = 0: P = 0.5
- If z = 2: P ≈ 0.88
- If z = -2: P ≈ 0.12

SIGMOID:



PROCESS:



Input Features → Linear Equation → $z = b_0 + b_1x$ → Sigmoid → Probability → Threshold → Class

DECISION:

If P >= 0.5:

Class = 1

If P < 0.5:

Class = 0

IMPORTANT: Logistic Regression does NOT directly predict a continuous value such as salary or weight. It predicts class probability and class labels.

Logistic Regression + Final Comparison

PYTHON:

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train)
prediction = model.predict(X_test)
probability = model.predict_proba(X_test)

print("Class:", prediction)
print("Probability:", probability)
```

EXAMPLE:

Probability of Pass = 0.82

Threshold = 0.5

0.82 >= 0.5

Therefore: **PASS**

Another example:

Probability of Pass = 0.30

0.30 < 0.5

Therefore: **FAIL**

FINAL COMPARISON

LOGISTIC REGRESSION	DECISION TREE	RANDOM FOREST
Core idea: Linear score + Sigmoid + Probability	Core idea: If-else rules + Recursive Splitting	Core idea: Multiple Decision Trees + Aggregation
Best for: Classification problems with a reasonably linear decision boundary and when probability/interpretability is important.	Best for: Non-linear relationships and interpretable rule-based decisions.	Best for: Strong general-purpose performance on many tabular classification/regression problems.
Main concept: SIGMOID	Main concepts: GINI · ENTROPY · INFORMATION GAIN	Main concept: MANY TREES + VOTING

QUICK COMPARISON

Algorithm	Main Idea
Logistic Regression	Probability using Sigmoid
Decision Tree	If/Else rules
Random Forest	Many Decision Trees + Voting

Task	Logistic Regression	Decision Tree	Random Forest
Classification	YES	YES	YES
Regression	NO	YES	YES

Property	Logistic Regression	Decision Tree	Random Forest
Non-Linear Relationships	Limited	Excellent	Excellent
Interpretability	Relatively easy	Very easy	More difficult

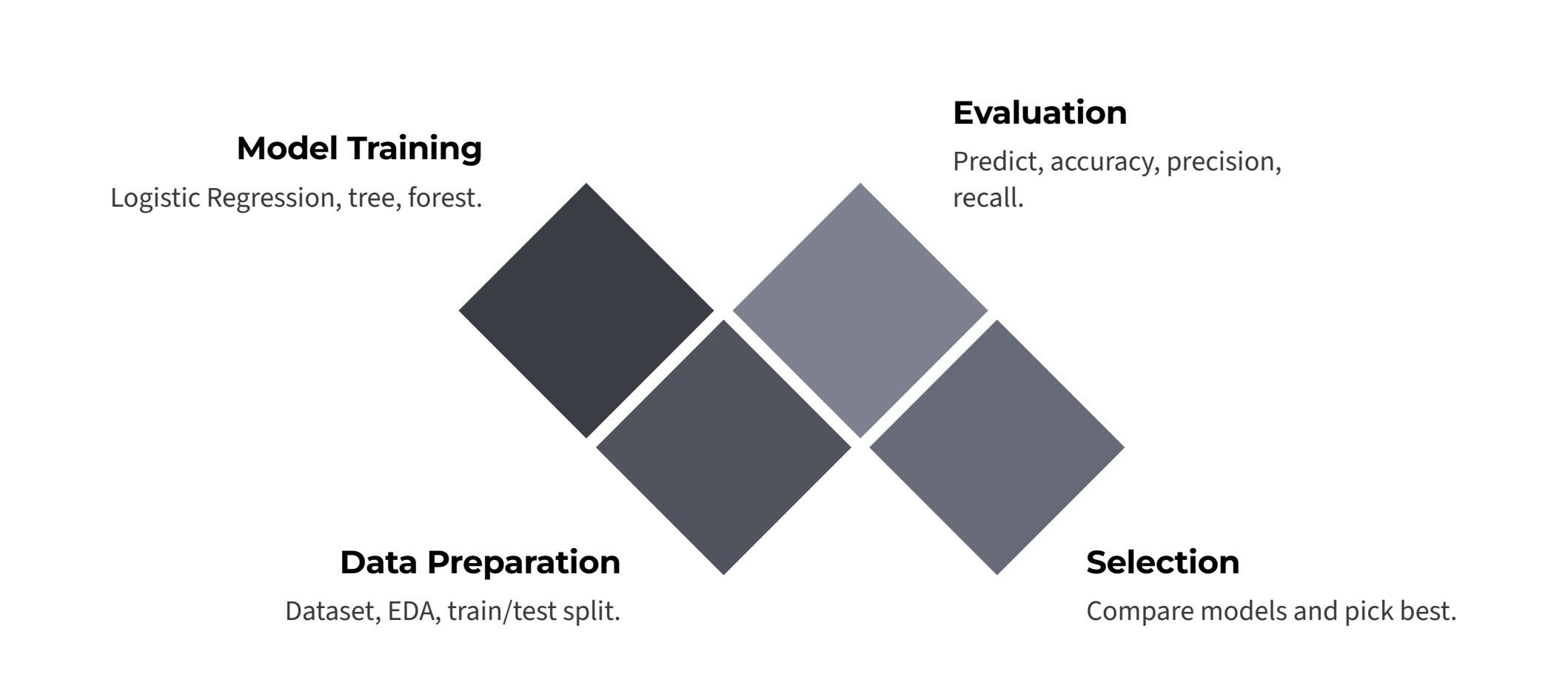
FINAL MEMORY TRICK

Logistic Regression
"How probable is this class?"

Decision Tree
"Which rule should I follow?"

Random Forest
"What do many trees collectively decide?"

PRACTICAL CLASS FLOW



Dataset → EDA → Train/Test Split → Logistic Regression → Decision Tree → Random Forest → Predict → Accuracy → Precision → Recall → F1 Score → Compare Models → Select Best Model